Vaultrune Registry - Cinderbound Custody Coffer, Warden's Field Reference

Vaultrune Registry - restricted to warranted field wardens. Issued with the recovery-bench handout.

For warranted field wardens of the Registry. A Cinderbound coffer is a pre-Sundering strongbox whose shell still answers to the old wards; the Registry refits each one with a modern control spine that speaks for them - a perfect face over the wrong bones. The coffer keeps a **sealed custody record** and surrenders it only to a boot the shell trusts: in service, a warden wakes the recovery ward with a service signet and reads the record out, and forcing the shell instead burns the record. This sheet is an operating reference for the spine, its wards, and the recovery bench. Every value is a working specification.

Essential constants

Parameter	Value
Core	Arm Cortex-M3 (Armv7-M, Thumb-2), 24 MHz
Internal SPI bus (flash + secure element)	4 MHz, mode 0 (CPOL=0, CPHA=0), MSB-first
Service UART	115200 8N1
Logic levels	3.3 V CMOS
Delay timebase	1 tick = 1 CPU cycle (24 MHz)
Frame checksum	CRC-16/CCITT-FALSE (poly 0x1021, init 0xFFFF)
Trigger origin	AUTH_DENIED CS_SE rising edge

Part 1 - Device and hookup

Processor and memory - the spine

The modern controller set behind the old face.

Region	Range	Access	Contents
Boot ROM	0x00000000+	R-X	measured-boot loader (disassembled as cinder_boot.elf)
SRAM	0x20000000+ (128 KiB)	RW	application RAM; the booted image loads at 0x20002000
SPI flash	2 MiB external, not memory-mapped	read over SPI (0x03)	two firmware slots

Flash slots: **production** image at `0x000000`, **recovery** image at `0x040000` (each up to `0x040000`).

Service header - the maintenance tap

The one non-destructive way into the coffer; the analyzer clamp returns these nine lines as a VCD.

Signal	Dir	Idle	Purpose
RESET	in	high	reset (active low)
RECOVERY_STRAP	in	low	asserts a recovery boot
CS_FLASH	out	high	flash chip-select (active low)
CS_SE	out	high	secure-element chip-select (active low)
SCK	out	low	SPI clock, mode 0

Signal	Dir	Idle	Purpose
MOSI	out	-	controller to peripheral
MISO	in	-	peripheral to controller
SE_IRQ	in	high	secure-element attention (active low)
GLITCH_OUT	bench	low	fault-injection pulse

Boot sequence

What the spine does on a recovery boot, and what shows on the bus.

#	Step	On the bus	Custody state
1	Power-on; read the recovery strap	-	BOOT_PCR = 0
2	Read and verify image manifests from flash	CS_FLASH: 0x03 reads	-
3	Wake the ward; read the NV counter	CS_SE: STARTUP, READ_NV_COUNTER	-
4	Low-counter recovery: issue a challenge, take a signet	GET_RECOVERY_CHALLENGE, VERIFY_RECOVERY_TICKET	AUTH_DENIED without a valid signet
5	On authorization, measure the image	EXTEND_BOOT_PCR, LOCK_MEASUREMENT	BOOT_PCR extended, then locked
6	Launch; the ward releases the record	OPEN_FACTORY_SESSION, UNSEAL, READ_FIFO	record released to the trusted firmware

Steps 5 and 6 are reached only on an authorized boot.

Part 2 - Formats and the secure element

Image manifest - the vendor seal (128 bytes, little-endian)

Every firmware slot is sealed under the Registry's own key.

`magic "CNDR"` · format version · header size (`0x0080`) · slot kind · payload size · SRAM load · entry offset · security counter · SHA-256 payload digest · ECDSA-P256 signature (`r||s`) · header CRC-32. The signature covers version, kind, sizes, load/entry, counter, and payload digest; the vendor **public** key is set into the boot ROM. A slot that does not verify does not boot - the seal cannot be re-cut in the field.

Recovery ticket - the service signet (144 bytes)

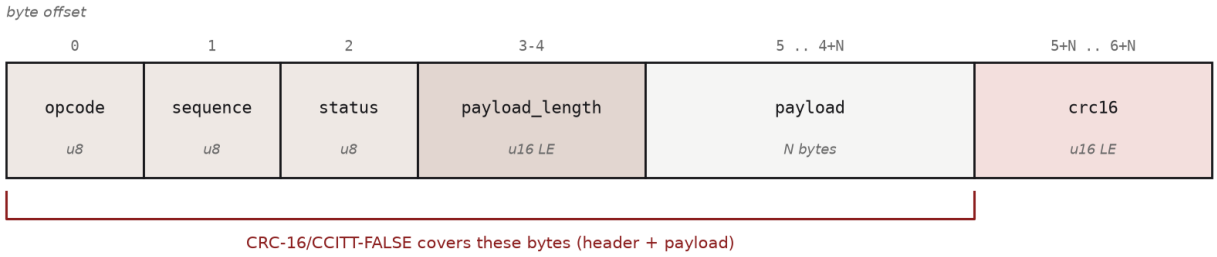
The token an auditor presents to wake the recovery ward for a single operation.

`magic "RCVT"` · version · flags · device UID · authorized image digest · authorized counter · challenge nonce · service op counter · ECDSA-P256 signature over the first 80 bytes. The ward checks the UID, the image digest and counter, the **current** challenge, and the signature against the service-signet **public** key. A signet that is malformed, stale, meant for another coffer, or wrongly signed is refused alike: every failure returns the same `AUTH_DENIED`, and nothing more.

Secure-element protocol - the ward's dialect

The ward answers on the internal bus in fixed frames.

`opcode:u8 | sequence:u8 | status:u8 | payload_length:u16le | payload | crc16:u16le`, checksummed per the constants table. The ward may hold the line with `0xff` wait bytes before it answers, and `READ_FIFO` returns at most 32 protected bytes per transaction.



Opcode	Command	Opcode	Command
0x10	STARTUP	0x15	LOCK_MEASUREMENT
0x11	READ_NV_COUNTER	0x20	OPEN_FACTORY_SESSION
0x12	GET_RECOVERY_CHALLENGE	0x21	UNSEAL
0x13	VERIFY_RECOVERY_TICKET	0x22	READ_FIFO
0x14	EXTEND_BOOT_PCR	0x23	GET_STATUS

Measured custody - the sealed record

The ward does not take a boot's word for what it is; it measures it. On power-on `BOOT_PCR = 0`, and the boot ROM extends the measurement once with `M = SHA256("CINDER-BOOT-v1" || slot_kind || counter_le32 || payload_size_le32 || payload_sha256)`, then locks it. The measurement is extend-only, only the boot locality may extend it, and `LOCK_MEASUREMENT` closes it until the next power cycle; application code cannot reach it. The sealed record's policy digest is held inside the ward and is not printed here.

The record's sealing key lives in the ward's tamper store and is **zeroized the instant the shell is breached** - the coffer's live mesh burns what it guards rather than surrender it. The internal bus is reached only through the service header, never by forcing the shell; once the ward releases the record, it travels under the factory session, not the mesh.

Part 3 - Fault timing and service access

Fault rig - the digital trigger

The recovery bench carries a fault rig for warden diagnostics. It watches the ward's response bytes for a raw masked pattern and, on the chosen `CS_SE` edge, fires a single pulse that omits exactly one instruction where it lands. You set the raw pattern, the raw mask, the edge, a 16-bit relative delay (fine ticks, 1 tick = 1 CPU cycle), and an 8-bit pulse width.

Timing reference - deriving the delay

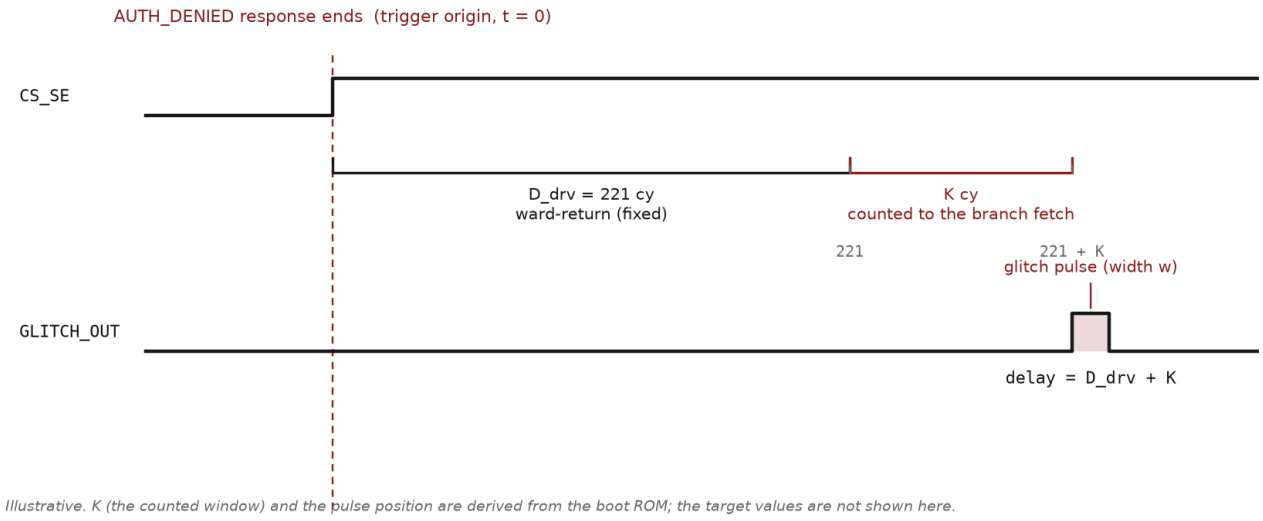
The delay is **counted, not guessed**, and this is the authoritative timing model for the bench. It is measured in CPU cycles (1 tick = 1 cycle at 24 MHz) from the triggering `CS_SE` edge - the rising (deassertion) edge that ends the `AUTH_DENIED` response transaction, the one whose response bytes

match the pattern. The bench is deterministic and prices instruction timing by the fixed convention below, so the interval is read straight off the disassembly:

Instruction class	Cycles
Data-processing / move / compare - any width, incl. .w and predicated (...eq ...ne ...cc ...cs) forms - and it / ite	1
Load / store (ldr str ldm stm push pop)	2
Taken branch, conditional or unconditional (b / b.n / a taken b<cc>)	3
bl (branch-and-link)	4
Not-taken conditional branch	1

Count along the **executed** control-flow path: follow taken branches, and within an `it` / `ite` block count only the leg whose condition holds. No interrupts fire during boot authorization, and there are no flash wait states in this window.

Ward-return cost. From the `AUTH_DENIED` `CS_SE` edge, the spine finishes the transfer, checks the response frame (the CRC-16 above, over the header), and returns from ticket verification into the boot flow in a fixed **D_drv = 221 cycles**. This covers the ward's frame-validation path and is given so the CRC loop need not be hand-counted; the delay is `D_drv` plus the cycles you count, under the convention above, from the first instruction after ticket verification returns up to - but not including - the enforcement branch's fetch.



Illustrative. K (the counted window) and the pulse position are derived from the boot ROM; the target values are not shown here.

Worked example (illustrative - not the target path). If the counted window were `ldr` (2) + `cmp` (1) + a not-taken `beq.n` (1) + a taken `b.n` (3), that is 7 cycles, so the delay would be `D_drv` + `7` = `228`. Count the real window from the disassembly the same way; the rig accepts a correctly-derived value within a small tolerance, so count carefully - a blind sweep of the delay space costs far more than the derivation.

Width calibration

Against a known, benign instruction sequence, a pulse width in roughly the `6..16` tick band produces a clean single-instruction effect; much narrower does nothing, and much wider resets the spine. This is a **calibration band, not an exploit-width guarantee** - it fixes the width, and says nothing about the target delay.

Recovery bench

The remote bench presents the coffer on three service lines; each line's address is assigned when the bench is spawned.

Line	Role
Rig control	power, strap, trigger, glitch, capture arm, status
Analyzer clamp	list and download captures (VCD.GZ)
Recovery-signet console	present a service signet

Drive them with `rig.py` (in the handout), which handles transport, capture download and verification, and the signet console. Captures are immutable gzip-VCD, envelope `CINDCAP1`, sealed with a SHA-256 integrity digest.